

Executive Summary

This repository implements an iOS app called *Swift Stock* (or *SwiftStock*), which allows a user to enter a stock ticker symbol, fetch real-time market data from the Yahoo Finance API (via RapidAPI), and display summary statistics and a 1-day intra-day price chart. The app is written in Swift with a UIKit storyboard UI, using CocoaPods for dependency management (SwiftChart for charts). Core components include a **HomeVC** for ticker input, a **DetailVC** for displaying stock data and drawing the chart, and Codable model classes for the JSON responses. Networking is done with `URLSession`, and custom view classes (`UIButtonStyle`, `ViewStyle`) provide styled UI elements. Overall, the intended flow is: user enters a symbol → app fetches summary and chart data → app updates labels and plots the chart. However, the app has minimal error handling and no persistence or background sync. Key concerns include exposed API keys, lack of tests/CI, and use of some deprecated APIs (like `keyWindow`).

Repository Structure and Key Files

The repo is a standard Xcode project with CocoaPods. Important parts (with example citations) include:

- **Root:** Contains `Podfile` (CocoaPods configuration) and the Xcode project/workspace. The `Podfile` specifies `platform :ios, '9.0'` and uses `use_frameworks!` with one pod: **SwiftChart** (MIT-licensed) ¹. No Carthage/SPM is used.
- **SwiftStock.xcodeproj / SwiftStock.xcworkspace:** Xcode project files.
- **SwiftStock/SwiftStock (App source):**
- **AppDelegate.swift, SceneDelegate.swift:** Boilerplate app lifecycle (SceneDelegate implies iOS13+). No custom code beyond default.
- **Info.plist:** Standard app configuration (bundle ID, icon, etc.), presumed default (target iOS 13+, since SceneDelegate is present). The minimum OS is set by the Podfile (iOS 9.0) but actual UI uses safe areas/scene management.
- **HomeVC.swift:** *Initial view controller*. Contains a text field (`txtF`) and a **Search** button. The button's action (`btnSubmit`) checks for a non-empty ticker and then instantiates `DetailVC` from the storyboard, passing the text as `parm` ². A regex in `UITextFieldDelegate` forbids non-letter input (enforcing A–Z characters only).
- **DetailVC.swift:** *Detail view controller*. It has IBOutlets for labels and a `Chart` view (from SwiftChart). The `parm` property receives the ticker. In `viewDidLoad`, it asynchronously calls `getApi(parm)` ³. That method shows a custom loading indicator and issues a GET to the Yahoo Finance summary endpoint using `URLSession` ⁴. On success it decodes a `StockResponse` (from `StockApiModel.swift`) and updates UI labels (`lblStockName`, `lblPrice`, `lblVolume`, `lblMarketCap`) on the main thread ⁵. It then calls `showOneDayChart(parm)` to fetch 1-day chart data. `showOneDayChart` issues another GET to the Yahoo 1-day chart endpoint ⁶, decodes a `ChartResponse` (from `ChartModel.swift`), and calls `initializeChart(model:)` on the main queue to plot the data. The `DetailVC` also implements `ChartDelegate` methods: on touch (`didTouchChart`) it shows the time and price at the touched point by adjusting a floating label; on

release (`didFinishTouchingChart`) it resets the label. A **Back** button calls `dismiss()` ⁷ . Error cases (missing data or API error) trigger a simple alert ⁸ .

- **ChartModel.swift:** Swift `Codable` structs generated for the chart JSON (symbols: `ChartResponse` , `ChartModel` , `Result` , `Indicators` , `Quote` , etc.) ⁹ . For example, `ChartResponse` has a `ChartModel` which contains `result: [Result]` , and each `Result` has `timestamp: [Int]` and `indicators.quote` array ⁹ .
- **StockApiModel.swift:** (Very large) Swift classes/structs for the Yahoo summary JSON. Only the `StockResponse` class with a `price: Price?` field is actually used ¹⁰ . The `Price` class has properties like `shortName: String?` , `regularMarketOpen: PostMarketChange?` , `regularMarketVolume: AverageDailyVolume10Day?` , `marketCap: AverageDailyVolume10Day?` ¹¹ . (Other fields are present but commented out.) This model is decoded in `getApi` to extract the stock name, price, volume, and market cap.
- **Utility/:** Contains styling classes. `UIButtonStyle.swift` is a custom `UIButton` subclass exposing `@IBInspectable` (corner radius, border width/color, gradient) ¹² . `VisualStyle.swift` is a similar subclass of `UIView` with border and shadow inspectables ¹³ . These are applied in Interface Builder to style buttons and containers. `CustomLoader.swift` defines a singleton `Indicator` that shows a full-screen `UIActivityIndicatorView` overlay (though it references the deprecated `UIApplication.shared.keyWindow`).
- **Assets.xcassets:** App icons and image assets.
- **Base.lproj/Main.storyboard:** The UI layout. Key scenes include *HomeVC* (initial scene) and *DetailVC*. For example, *HomeVC* has a label “Swift Stock”, a text field for the symbol, and a styled **Search** button linked to `btnSubmit:` ¹⁴ ¹⁵ . *DetailVC*’s scene has a title bar (“Swift Stock”), a Back button connected to `btnBack:` , a colored styled container (`VisualStyle`) with labels for *Price* , *Volume* , *Market Cap* (and blank value labels), and a `Chart` view (200pt tall, orange background) for the `SwiftChart` graph ¹⁶ . Outlets in *DetailVC* match these: e.g. `lblPrice` connected to the “price” label, and `chart` to the Chart view. The storyboards tie UI elements to code (see [45†L939-L947] and [50†L1546-L1554]).

Table: Key components and files.

Component / File	Purpose & Contents	Example Citation
<code>Podfile</code>	CocoaPods setup (sets iOS 9.0+, adds <code>SwiftChart</code> pod) ¹	[82†L277-L285]
<code>AppDelegate.swift</code>	App lifecycle (default code, no custom logic)	–
<code>SceneDelegate.swift</code>	Scene/window setup (iOS13+; default)	–
HomeVC.swift	Main UI: text field + Search button. Validates input and presents <code>DetailVC</code> with ticker ² .	[53†L395-L403]

Component / File	Purpose & Contents	Example Citation
DetailVC.swift	Detail UI: fetches data & chart. In <code>viewDidLoad</code> calls <code>getApi(parm)</code> ³ . Uses <code>URLSession</code> to GET summary JSON ⁴ , updates labels (stock name, price, volume, cap) ⁵ , then calls <code>showOneDayChart</code> to fetch chart JSON ⁶ and plots it. Implements chart touch callbacks.	[57†L1157-L1165] [57†L1234-L1242]
ChartModel.swift	Codable structs for chart API JSON (<code>ChartResponse</code> , <code>ChartModel</code> , <code>Result</code> , <code>Indicators</code> , <code>Quote</code>) ⁹ . Used to decode intra-day price data.	[60†L412-L420]
StockApiModel.swift	Codable classes for stock summary JSON. <code>StockResponse</code> holds a <code>Price?</code> ¹⁰ , which contains fields (shortName, regularMarketOpen, volume, marketCap) ¹¹ . Used to parse API response in <code>getApi</code> .	[65†L1974-L1983] [68†L2665-L2673]
Utility/ UIButtonStyle.swift	Custom UIButton subclass with IBInspectable properties (corner radius, border color/width, gradient, shadow) ¹² . Used to style the Search button and others.	[85†L580-L589]
Utility/ViewStyle.swift	Custom UIView subclass with inspectable border and shadow properties ¹³ . Used for the container behind the stock stats.	[86†L600-L608]
Base.lproj/ Main.storyboard	Interface Builder UI layout: scenes for HomeVC and DetailVC, defines labels, text field, buttons, and the SwiftChart view ¹⁷ ¹⁶ . Ties UI to code via outlets/ actions.	[45†L939-L947] [50†L1546-L1554]

Build and Run Instructions

The project uses **Xcode** and **Swift** (versions not explicitly stated). Given the code (e.g. use of SceneDelegate, Swift 5 syntax, and CocoaPods with `use_frameworks!`), it likely requires Xcode 11 or later (iOS 13+ SDK) and Swift 5. The Podfile sets `platform :ios, '9.0'` ¹, but the actual UI uses iOS 13+ features (scene, safe areas). To build: 1. Ensure **CocoaPods** is installed.

2. Run `pod install` in the repo root (the included `Podfile` references `SwiftChart`) ¹.

3. Open the `.xcworkspace` in Xcode. (The workspace includes Pods.)

4. Select a target device (probably iOS 13.0+).

5. Build and run. The app uses SwiftChart (MIT license) as its only third-party dependency ¹. No other CocoaPods/SPM/Carthage libs are used.

Dependencies:

- [SwiftChart](#) (via CocoaPods) – line/area chart library (MIT license) ¹.
- No persistent storage or databases; networking uses Apple's Foundation frameworks (URLSession).
- No specific frameworks (like Combine, CoreData) are used.

Detailed Component Analysis

UI Layer (Storyboards & Views)

- **Home Screen (HomeVC):** A label "Swift Stock", a text field (`txtF`) for input, and a *Search* button (`UIButtonStyle`). All arranged in a simple vertical layout. The button has custom styling (rounded corners, border) via `UIButtonStyle.swift` ¹². Input is restricted to letters only (see delegate regex in code ¹⁸). If the text field is empty, the button does nothing (no alert shown).
- **Detail Screen (DetailVC):** Contains at top a title label and a *Back* button. Below is a colored container view (`ViewStyle` with border/shadow) that holds three horizontal stacks of label pairs: "Stock price:", "Volume:", "Market cap:" on the left, and empty labels (to be filled) on the right. Those right-side labels are connected to `lblPrice`, `lblVolume`, `lblMarketCap` in code (via outlets). The container has `cornerRadius=8`, `borderWidth=1` set in IB. Under this, a **SwiftChart** view (class=Chart) of height 200 displays the price-time area chart ¹⁶. A floating UILabel (`lblStockName`) above the chart is used to show the selected time (initially empty), and another label (`closelabel`) to show the selected price (populated on touch). Constraints are set so the chart fills most of the view width with margins 20/20. The storyboard IDs ("HomeVC", "DetailVC") connect scenes to the custom Swift classes.

View Controllers

- **HomeVC.swift:** Implements `UITextFieldDelegate` to allow only letters (regex `".*[^A-Za-z].*"` rejects digits/symbols) ¹⁸. The `btnSubmit(_:)` action does:
 - Checks if `txtF.text` is empty; if so, returns immediately ¹⁹ (no feedback shown).
 - Instantiates `DetailVC` from "Main" storyboard by its identifier ("DetailVC") ²⁰.
- Sets `vc.parm = txtF.text!` and presents it full-screen.
(The `showAlert()` stub is present but not called; invalid tickers are only handled in DetailVC.)
- **DetailVC.swift:** Main logic happens here. Key points:
 - Properties/IBOutlets: `parm: String` (ticker passed from Home), labels (`lblStockName`, `lblPrice`, `lblVolume`, `lblMarketCap`, `label`, `closelabel`), the `Chart` view, and a constraint `labelLeadingMarginConstraint` for floating label positioning.
 - **viewDidLoad:** Saves the initial left margin of the floating label, then after a 0.1s delay calls `getApi(parm)` ²¹. (The delay ensures the UI is laid out first.)
 - **getApi(_):** Shows the loading indicator (`Indicator.sharedInstance.showIndicator()`), then forms a GET request to Yahoo Finance's "stock summary" endpoint via RapidAPI ⁴. It uses hard-

coded HTTP headers including a RapidAPI key (security concern) ²². The URL is

`.../get-summary?symbol=SYM®ion=US`.

- The request is sent asynchronously (`URLSession.dataTask`). On response, the code dispatches back to the main thread to hide the indicator ²³, and then tries to decode the JSON into `StockResponse` via `JSONDecoder` ²⁴.
- If decoding succeeds, it extracts `model.price?.regularMarketVolume?.fmt`, `regularMarketOpen?.fmt`, `shortName`, `marketCap?.fmt` (all `String?`) ²⁵. If a valid price is found, it updates the UI labels on the main queue ²⁶. Specifically:

```
self.lblStockName.text = stockName
self.lblPrice.text = price
self.lblVolume.text = volume
self.lblMarketCap.text = marketCap
```

- Then it calls `showOneDayChart(parm)` ²⁷. If `price` or `volume` is nil/empty, it instead calls `showAlert()` ²⁸. Similarly, if decoding fails, it also shows the alert ⁸.
- **showOneDayChart(_):** Similar pattern for the chart data: shows indicator, builds a GET to `.../get-chart?interval=5m&symbol=SYM&range=1d®ion=US` ²⁹, and starts a `URLSession` task. On response, it decodes into `ChartResponse` and then on the main queue calls `initializeChart(model:)`.
- **initializeChart(model:):** Uses the model's `timestamp` and `close` arrays to build a `SwiftChart` `ChartSeries`. It assembles an array of `(date, close)` points, extracts month labels for the X-axis, and then configures the chart view (sets line width, label font, disables X-grid/labels, aligns Y-labels right, etc.) ³⁰. It sets `chart.minY/maxY` from the data range, and calls `chart.add(series)` to draw it.
- **Chart Delegate:** The `didTouchChart(_:indexes:x:left:)` method responds to user touches on the chart. It gets the touched index and value, formats the value to 2 decimals, formats the date from the timestamp, and sets the floating `label.text` to the time and `closeLabel.text` to the price ³¹. It then adjusts `labelLeadingMarginConstraint.constant` so the label moves with the touch but doesn't go off-screen ³². `didFinishTouchingChart` clears the label once touch ends.
- **Error Handling:** Very minimal. Network errors or JSON parsing errors are only printed to console. The user sees an alert *only* if the summary price or volume is missing (interpreted as invalid ticker) ²⁸ ⁸. No retry or detailed error messages are provided.
- **Indicator:** A custom singleton `Indicator.sharedInstance` is used to show/hide a loading spinner over the full screen while network requests are in flight (called at the start and end of each fetch).

Data Models (Codable structs)

- **ChartModel.swift:** Defines the structures matching Yahoo's chart JSON. The top-level `ChartResponse` has a `chart: ChartModel?` field ⁹. `ChartModel` has `result: [Result]?` and an error field. Each `Result` has `[Int] timestamp` and an `Indicators` object. `Indicators` has an array `[Quote] quote`. `Quote` contains arrays of `Double` for `close`, `high`, `low` and `[Int] volume` ³³. These classes/structs allow `JSONDecoder` to map the nested JSON into Swift types. (Other parts of the JSON are ignored by commenting out fields.)

- **StockApiModel.swift:** A large set of classes for the Yahoo summary. Only `StockResponse` (top-level) and `Price` (nested) are used. `StockResponse` is a `class Codable` with one property `price: Price?` ¹⁰. The `Price` class has properties relevant for our use:

- `shortName: String?` (the company name),
- `regularMarketOpen: PostMarketChange?` (opening price),
- `regularMarketVolume: AverageDailyVolume10Day?` (volume),
- `marketCap: AverageDailyVolume10Day?`.

These types (`PostMarketChange`, `AverageDailyVolume10Day`) are themselves `Codable` wrappers for numeric or string values. In `DetailVC`, after decoding, we use e.g.

`model.price?.regularMarketOpen?.fmt` to get a formatted string price ²⁵. (All other fields in `StockResponse` are commented out.)

Networking and Concurrency

All network calls use `URLSession.shared.dataTask`. By default, `dataTask` runs the request on a background thread. The code correctly wraps UI updates in `DispatchQueue.main.async` to avoid threading issues ²³ ²⁶. The chart initialization is also called on the main queue. There are no explicit concurrency mechanisms beyond these closures. Notably, `getApi(_:)` does **not** use `[weak self]`, meaning the closure could potentially capture `self` strongly until completion (though the VC is dismissed on back so this is low-risk). In `showOneDayChart`, the closure uses `[weak self]` but then force-unwraps `self!` inside, which could crash if `self` became nil. In general, network tasks are serial and relatively lightweight for a single ticker query.

No background fetch or multi-threading beyond this. The chart drawing runs on the main thread; since it's a small 200-point chart, performance is not an issue. There is **no offline support or caching**: each view appearance triggers new API calls. No use of `URLCache`, no persistence (`UserDefaults/CoreData`), and no background refresh tasks.

Data Flow and Sequence

```
sequenceDiagram
    actor User
    User->>HomeVC: Enter ticker & tap Search
    HomeVC->>DetailVC: instantiate with parm=ticker 2
    DetailVC->>DetailVC: viewDidLoad (asyncAfter 0.1s) 3
    DetailVC->>YahooAPI: GET /stock/v2/get-summary?symbol=SYM 4
    Note right of YahooAPI: Returns JSON (StockResponse)
    YahooAPI-->>DetailVC: JSON data
    DetailVC->>DetailVC: decode StockResponse 24
    DetailVC->>DetailVC: update UI labels (name, price, volume, cap) 26
    DetailVC->>DetailVC: call showOneDayChart() 27
    DetailVC->>YahooAPI: GET /stock/v2/get-chart?symbol=SYM&range=1d 6
    Note right of YahooAPI: Returns JSON (ChartResponse)
    YahooAPI-->>DetailVC: JSON data
```

```
DetailVC->>DetailVC: decode ChartResponse
DetailVC->>DetailVC: initializeChart(model:) 34
DetailVC->>UI: plot chart
```

This depicts the core flow: user action triggers two sequential network calls. Data is decoded into models (`StockResponse` , `ChartResponse`), then the app updates its view (labels and chart). Touching the chart (not shown above) triggers `didTouchChart` , which formats and displays the touched data point.

Dependency and Security Analysis

Third-party libraries: The only external library is **SwiftChart** (added via CocoaPods) ¹ . SwiftChart is MIT-licensed ³⁵ , so no restrictive license issues. There are no other major libraries (no Alamofire, no JSON libs besides Foundation).

License and Notices: The custom UI classes (`UIButtonStyle` , `ViewStyle`) appear to be taken from online sources (comments credit an "ARDevelopers 2018"). No license header is included, so their license is unknown. This may be a concern if redistributing, but for analysis we note their origins.

API Keys and Secrets: The code contains hard-coded RapidAPI keys (e.g. `"x-rapidapi-key": "65fef0ace7mshe7...874f"` in `getApi`) ²² . Embedding API keys in source code is a security risk – anyone with the binary or source can extract the key. Ideally these should be stored securely (e.g. in the keychain, or injected at build time via environment). Also, the key shown appears to be a default or free-tier key. If this were a real product, revoking such an exposed key would be necessary.

Networking/security: Calls use HTTPS endpoints (`https://apidojo-yahoo-finance-v1.p.rapidapi.com/...`), which is good. There is no certificate pinning or advanced security, but standard HTTPS suffices here. The app does not use App Transport Security exemptions (the domain is HTTPS).

Vulnerable Patterns: Aside from the API key issue, other concerns include:

- The use of `UIApplication.shared.keyWindow` in `CustomLoader` (deprecated as of iOS 13) – not a security flaw but a code maintenance issue.
- No input sanitization beyond the regex; unusual ticker symbols (with `-` or numbers) might behave unpredictably.
- The empty dispatch `DispatchQueue.main.asyncAfter(deadline:)` is a workaround; if `parm` is already empty, nothing happens (no crash, but no feedback).
- No SSL or JSON validation is done beyond `JSONDecoder` ; a malformed response would throw and just be printed, not shown to user.
- No rate-limiting or request deduplication: rapid repeated presses would send multiple calls (though UI blocks if text empty).

Vulnerabilities of Dependencies: SwiftChart is open source and widely used; it has no known major vulnerabilities. CocoaPods itself is mature. The only risk might be if SwiftChart had a bug, but none are evident here.

Tests Coverage and CI Configuration

There are **no test files or CI config** in the repository. No unit tests, UI tests, or continuous integration scripts. Code coverage is therefore 0%. For a production project, this is a major gap. Testing could include:

- Unit tests for model decoding (`ChartResponse` , `StockResponse`) using sample JSON to ensure parsing works as expected.
- Integration tests for `DetailVC` : mocking network calls (or using a stub server) to test UI updates and error handling.
- UI tests for the text input and button behavior.
- A CI workflow (e.g., GitHub Actions or Jenkins) to run these tests on each commit, plus lint/static analysis.

Without tests, regression bugs are likely (e.g. if API changes, or after refactoring). The absence of CI means nothing enforces code quality or automatic builds.

Runtime Behavior and Edge Cases

- **Normal case:** Enter a valid ticker (e.g. "AAPL"), tap Search. The app transitions to `DetailVC`, fetches the summary, displays the company name, price, volume, and market cap, and then shows a 1-day price chart. Touching the chart displays that time & price.
- **Invalid/empty input:** If the text field is empty, the app simply does nothing (ignores the button press) ¹⁹ . No alert is shown on `HomeVC` for invalid input. If the user enters non-letters (e.g. "123"), the delegate prevents it (the regex blocks any non-A-Z character) ¹⁸ .
- **Ticker not found or unsupported:** If the API returns no data (e.g. an unrecognized symbol), `regularMarketOpen` or `volume` will likely be nil. In that case, the code calls `showAlert()` ²⁸ ⁸ which displays "Warning! Incorrect Stock Ticker". (This is the only user-facing error message.) The chart will not load. However, no distinction is made between "ticker doesn't exist" vs. "network error" vs. "format error" – all these end up printing to console and often showing the same alert.
- **Network errors:** If the network request fails (no internet, timeout), the error is just printed (no user alert). The loading indicator is still hidden on failure due to the `DispatchQueue.main.async` call ²³ , but the user sees no data or feedback.
- **UI/Threading:** All UI updates happen on the main thread. The network tasks correctly use background threads (URLSession defaults). There is a slight risk: in `showOneDayChart` , the closure is `[weak self]` but uses `self!` (force unwrap) to set `chartModel` . If `self` had been deallocated (unlikely since the VC is still on screen), it would crash.
- **Memory:** No obvious leaks – closures use `[weak self]` in one case, but not in the first API call (although that VC won't be deallocated until dismissed). The large data models (`StockApiModel.swift`) might occupy memory, but are released after decoding. No caching means memory resets on each new fetch.
- **Chart Data Range:** The chart is hard-coded to show 1 day of 5-minute data. If a stock trades less frequently (low volume), the chart might look flat or empty. If the market is closed (e.g. after hours), the Yahoo chart API may return an empty series, potentially causing a crash (unhandled).
- **Concurrency Edge Cases:** The 0.1s delay in `viewDidLoad` is a hack – if `parm` is set to empty (it shouldn't be), `getApi` is still called after delay but immediately returns. If the back button is pressed very quickly, the async calls may still complete and try to update a dismissed VC (though the alert uses `self.present` , which would fail silently if `self` is gone).

Overall, the runtime behavior mostly “works” for simple use, but robustness is low. Many edge cases are not user-friendly, and failures tend to be silent.

Performance and Scalability

- **Performance:** The app makes at most two quick HTTP requests per search. These are small JSON payloads (summary and 1-day chart). Parsing 200 data points and rendering an area chart is lightweight on modern devices. UI updates are minimal (a few labels). Thus, performance on iPhones/iPads should be fine. One potential issue: using the default `JSONDecoder` on the main queue just before UI updates (currently they decode on the background thread, so it’s acceptable). There’s no large image loading or heavy loops.
- **Memory:** The largest memory usage is likely the parsed JSON models. The stock summary model (700+ lines) might allocate dozens of objects, but these are ephemeral. The chart data is at most ~300 points (5 min * 6.5 hours trading = ~78 points) – trivial. There is no memory caching and no large data sets, so memory use is low.
- **Scalability:** The app is single-user, single-stock at a time. There is no built-in support for multiple stocks or historical ranges beyond 1 day. Scaling to many users would require migrating the API calls to a backend. Within the app, one could imagine extending to 1-week, 1-month charts, or an index vs multiple tickers. As-is, it only handles one request at a time; tapping search quickly multiple times would spawn overlapping requests. There is no rate-limiting or queuing. The UI could be enhanced to disable the Search button while loading.

Suggested Improvements and Actionable Fixes

Below are prioritized fixes and enhancements, with rough effort estimates (Small=1–3 days, Medium=1–2 weeks, Large=1+ month).

1. **Remove Hardcoded API Keys (Security Fix, Medium):** Do **not** embed RapidAPI keys in code. Instead, use a secure method (e.g. store in Keychain or fetch from a secure config at runtime). At minimum, move it to a configuration file excluded from source control. This is critical for production. (Effort: Medium, ~1–2 days)
2. **Improve Error Handling (Small):** Currently invalid tickers or network failures are almost silent. Add alerts or UI indications for network errors (e.g. “Network error, please try again”). Handle HTTP status codes and missing fields more gracefully ²⁸. (Effort: Small, ~1 day)
3. **Validate and Normalize Input (Small):** Allow numbers and symbols typically in tickers (e.g. “BRK-A”, “TSLA”). The regex currently blocks digits; consider a whitelist approach or trim whitespace ¹⁸. Inform the user immediately if input is invalid. (Effort: Small)
4. **Deprecation Cleanup (Small):** Update `CustomLoader.swift` to avoid `keyWindow` (use `windowScene` on iOS13+). This will ensure the loading indicator appears correctly on all scenes. (Effort: Small)
5. **Add Unit Tests (Medium):** Write unit tests for JSON parsing (`ChartResponse`, `StockResponse` with known JSON samples) and for simple ViewController logic (maybe using XCTest and mocks). Without refactoring, this is straightforward. (Effort: Medium, ~1 week)
6. **Set Up CI (Small):** Add a GitHub Actions or other CI workflow to run builds and tests on push. This requires only a config file. (Effort: Small)

7. **Chart Flexibility (Medium):** Allow selecting different time ranges (e.g. 5d, 1y) using Yahoo API parameters. This involves UI changes (segmented control) and parsing potentially larger data sets. (Effort: Medium)
8. **Caching & Persistence (Large):** Cache recent queries or chart data locally (CoreData or file) to speed up repeat views. Possibly add a “Favorites” feature. (Effort: Large)
9. **Refactor Networking (Medium):** Currently using `URLSession` manually. Could switch to modern `async/await` (Swift 5.5+) for cleaner code, or use `URLSession.shared.data(for:)` convenience APIs. Also consider a network layer class. (Effort: Medium)
10. **Improve UI/UX (Medium):** Indicate loading states clearly (disable button during fetch), validate text with placeholders, and style the chart tooltip more visibly. (Effort: Medium)

Each of these improvements would make the app more robust, maintainable, and user-friendly. The highest priority are security fixes (API keys) and basic error handling. Adding tests and CI will pay off for future development.

```
classDiagram
    StockResponse --> Price : price
    Price : String shortName
    Price : PostMarketChange regularMarketOpen
    Price : AverageDailyVolume10Day regularMarketVolume
    Price : AverageDailyVolume10Day marketCap

    ChartResponse --> ChartModel : chart
    ChartModel --> Result : result[]
    Result : Int[] timestamp
    Result --> Indicators : indicators
    Indicators --> Quote : quote[]
    Quote : Double[] close
    Quote : Double[] high
    Quote : Double[] low
    Quote : Int[] volume
    Quote : Double[] open
```

The above class diagram shows the main data relationships. `StockResponse` holds a `Price` (company name, volume, etc.). `ChartResponse` holds a `ChartModel` with a list of `Result` entries, each of which has timestamps and an array of `Quote` data (closing prices, etc.) which the app uses to plot the chart.

Sources: Implementation details are drawn directly from the repository code, e.g. `HomeVC.swift` and `DetailVC.swift` ² ⁴, and from the Podfile ¹. Use of `SwiftChart` is confirmed by the Podfile ¹ and storyboard custom class ¹⁶. All cited code is from the GitHub repo’s files. Official documentation (Apple Developer) was consulted implicitly for concurrency/`URLSession` behavior.

¹ Swift-Stock/Podfile at main · rimanov/Swift-Stock · GitHub
<https://github.com/rimanov/Swift-Stock/blob/main/Podfile>

2 15 18 19 20 **Swift-Stock/SwiftStock/HomeVC.swift** at main · rimanov/Swift-Stock · GitHub

<https://github.com/rimanov/Swift-Stock/blob/main/SwiftStock/HomeVC.swift>

3 4 5 6 7 8 21 22 23 24 25 26 27 28 29 30 31 32 34 **Swift-Stock/SwiftStock/DetailVC.swift** at main · rimanov/Swift-Stock · GitHub

<https://github.com/rimanov/Swift-Stock/blob/main/SwiftStock/DetailVC.swift>

9 33 **Swift-Stock/SwiftStock/ChartModel.swift** at main · rimanov/Swift-Stock · GitHub

<https://github.com/rimanov/Swift-Stock/blob/main/SwiftStock/ChartModel.swift>

10 11 **Swift-Stock/SwiftStock/StockApiModel.swift** at main · rimanov/Swift-Stock · GitHub

<https://github.com/rimanov/Swift-Stock/blob/main/SwiftStock/StockApiModel.swift>

12 **Swift-Stock/SwiftStock/Utility/UIButtonStyle.swift** at main · rimanov/Swift-Stock · GitHub

<https://github.com/rimanov/Swift-Stock/blob/main/SwiftStock/Utility/UIButtonStyle.swift>

13 **Swift-Stock/SwiftStock/Utility/ViewStyle.swift** at main · rimanov/Swift-Stock · GitHub

<https://github.com/rimanov/Swift-Stock/blob/main/SwiftStock/Utility/ViewStyle.swift>

14 16 17 **Swift-Stock/SwiftStock/Base.lproj/Main.storyboard** at main · rimanov/Swift-Stock · GitHub

<https://github.com/rimanov/Swift-Stock/blob/main/SwiftStock/Base.lproj/Main.storyboard>

35 **SwiftChart** on CocoaPods.org

<https://cocoapods.org/pods/SwiftChart>